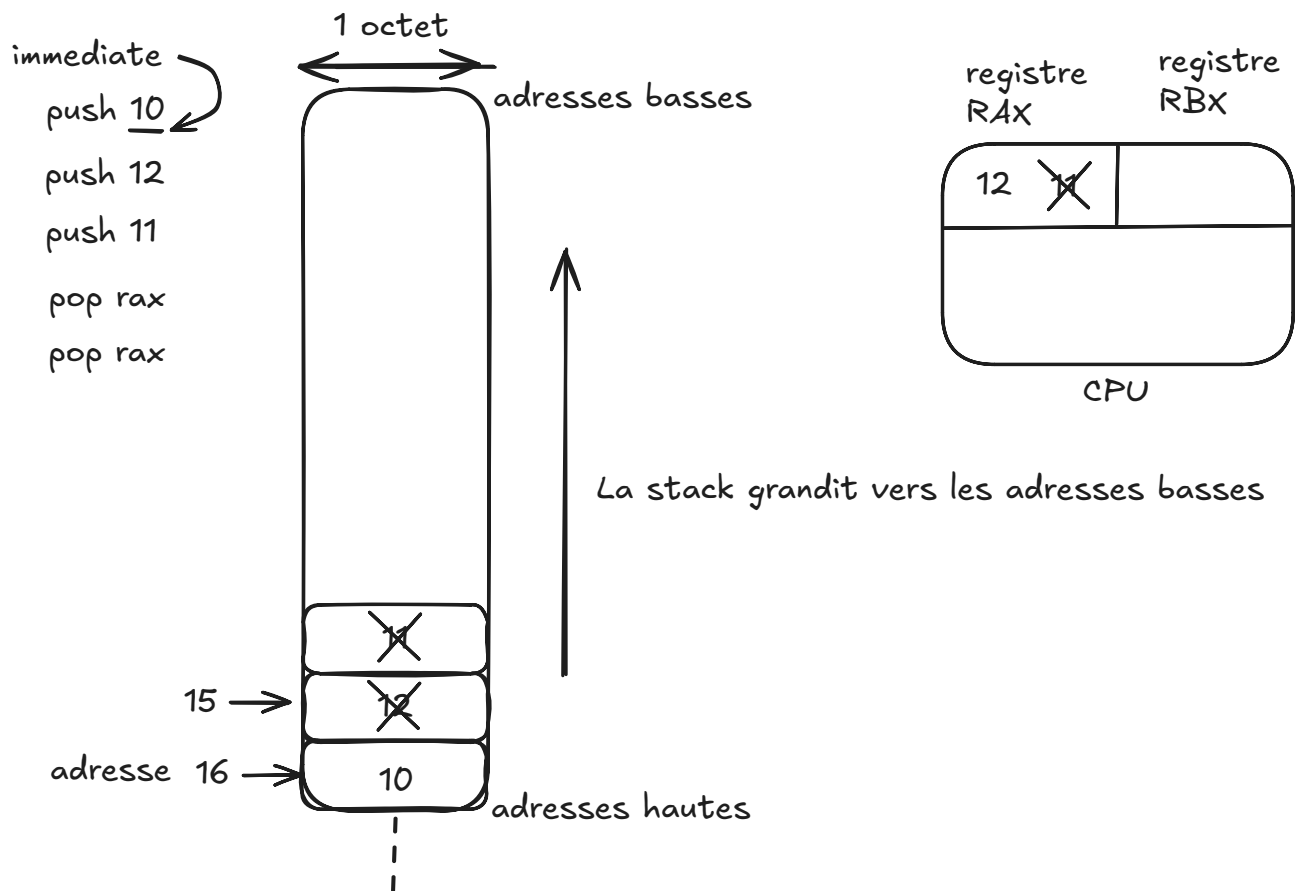


10-03-2025

Ré-explication de la stack



Pour voir la RAM libre (sur Linux), il faut écrire `free -h` → le -h est pour que ce soit lisible pour un humain

	total	used	free	shared	buff/cache	available
Mem:	7.6Gi	1.2Gi	6.4Gi	2.5Mi	235Mi	6.4Gi
Swap:	2.0Gi	0B	2.0Gi			

Chaque octet dans la RAM à son adresse, les registres n'ont pas d'adresse

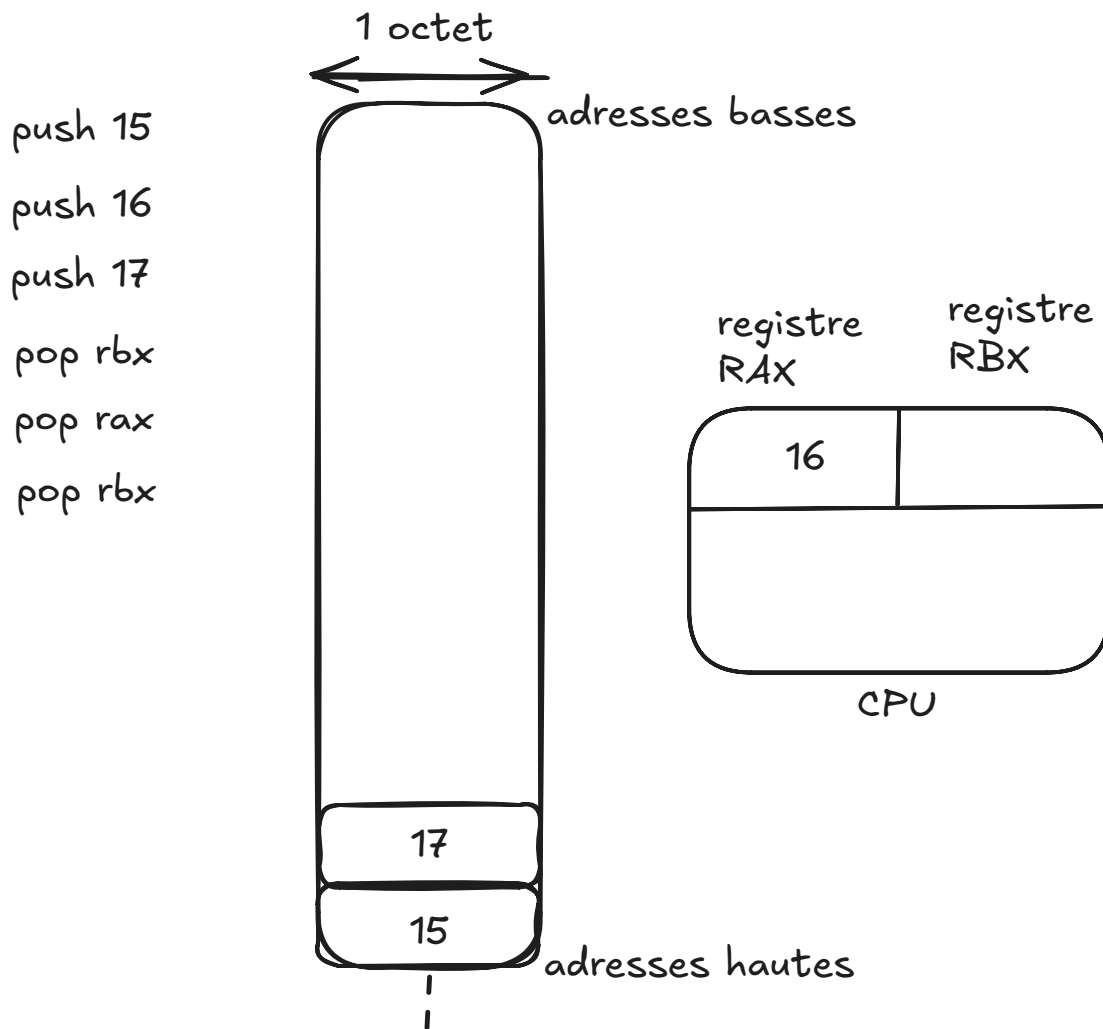
à t on le droit d'écrire partout dans la RAM ?

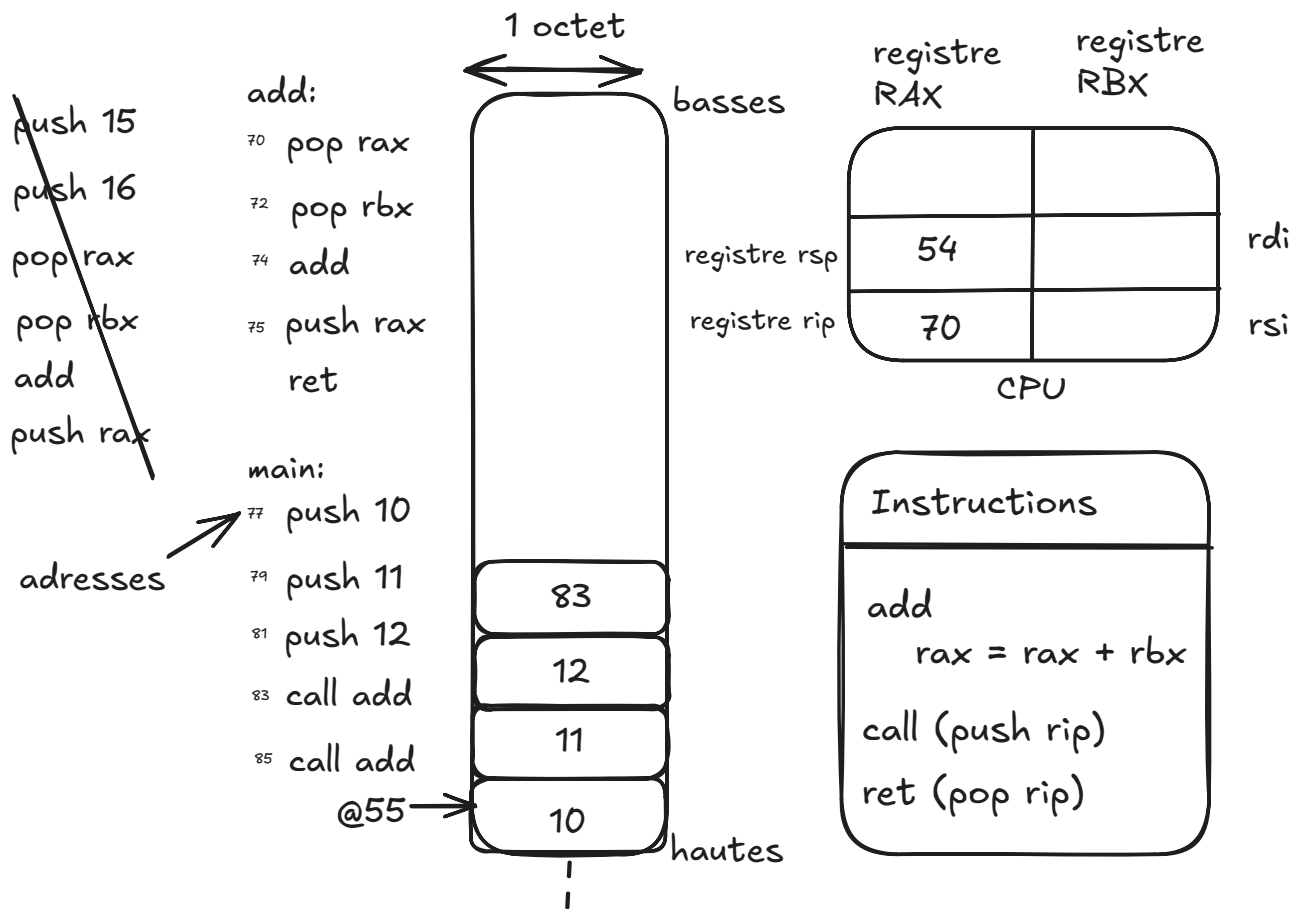
En théorie oui mais en pratique non

Pourquoi l'implémentation de fib qu'on a fait est si lente ?

Pour faire fib(6), il doit faire fib(5) + fib(4), pour faire fib(5) = fib(4) + fib(3), etc...

Comment faire pour enlever le 16 qui est entre 15 et 17 ?





Assembly

```

1  global main
2  add:
3      push    rbp    ;prologue
4      mov rbp, rsp    ;prologue
5      add rdi, rsi    ; -> rdi = rdi + rsi
6      mov rax, rdi
7      pop rbp        ;epilogue
8      ret            ;epilogue
9
10 main:
11     push    rbp
12     mov rbp, rsp
13     mov rdi, 11     ;1er argument
14     mov rsi, 12     ;2eme argument
15     call    add
16     mov rax, 0      ;return 0 donc si le programme réussit
17     pop rbp
18     ret

```

On veut le compiler

```
nasm -g -f elf64 add.s -o add.o
```

fichier nasm

-g → pour avoir des symboles de debug

à la fin, cela nous fait un fichier .o qui n'est pas un programme

```
gcc -g3 add.o -o add pour avoir le fichier en executable  
add est l'executable
```

sans `mov rax, 0`, quand on execute le programme, il crache et nous donne le nombre directement alors qu'avant on le voyait pas

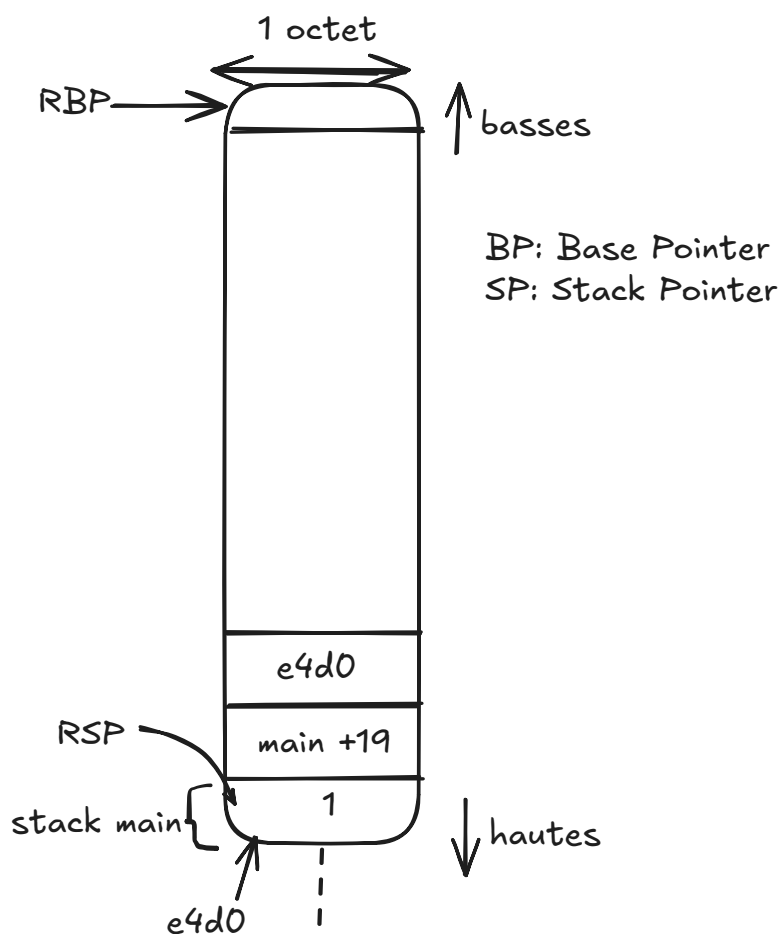
`x/20i` → explore 20 instructions à partir de cette adresse

`si` → step instruction

(gdb) → debugger d'Assembleur et de C

Du gros au petit bout (de l'oeuf) → Big endian - gros boutiste

Du petit au gros bout → Little endian - petit boutiste



```
x/1a $rsp
```

```
sudo apt install nasm  
sudo apt install gdb
```

```
1 nasm -g -f elf64 add.s -o add.o
```

```
2 gcc add.o -o add
3 gdb add
4 b main
5 r
6 exit
7 ./add
8 echo $?
```

Maintenant le faire avec trois valeur

Assembly

```
1 global main
2 add:
3     push    rbp
4     mov rbp, rsp
5     add rdi, rsi ; -> rdi = rdi + rsi
6     mov rax, rdi
7     pop rbp
8     ret
9
10 main:
11     push    rbp
12     mov rbp, rsp
13     mov rdi, 5 ;1er argument
14     mov rsi, 9 ;2eme argument
15     call    add
16     mov rsi, 10
17     call    add
18     pop rbp
19     ret
```

Traduction en C

C c

```
1 int add(int a, int b) {
2     return a + b;
3 }
4
5 void main(void) {
6     add(4, 8);
7 }
```

C c

```
1 int main (void) {
2     char un = '1';
3     char deux = '2';
```

```
4     char *hello1 = "Hello world\n";
5     char *hello2 = "Hola!\n";
6     print_string(hello1);
7     print_string(hello2);
8 }
```

C c

```
1 void print_string(char *address_of_first_char) {
2     while(1){
3         if (*address_of_first_char == 0)
4             return;
5         write(1, //sortie = stout
6             address_of_first_char, // adresse du premier caractère à afficher
7             1); //nombre de caractère à afficher
8         address_of_first_char += 1;
9     }
10 }
```